Technical Analysis of `memvar.h`: A History-Tracking Value Wrapper for C++20

Rel. v.1.2.0

October 8, 2026

Abstract

This report presents a detailed, code-derived technical analysis of `memvar.h`, a single-header C++20 library implementing two class templates, `mv::memvar<T>` and `mv::memvarTimed<T,Time,Clock>`, which wrap a scalar-like value together with a bounded history of its previous values. The analysis was produced strictly from the source code, independent of any comments present in the file, and every claim of behavior in this report — including the identified defects — was confirmed by compiling and executing small, self-contained C++20 programs against the header with GCC 13. The report covers the motivation and design rationale as can be inferred from the code structure, the type-constraint system built from `concepts`, the internal storage strategy based on `std::deque`, the complete public interface of both classes, the free-function operator overload set, worked usage examples, and a systematic catalogue of edge cases, limitations, and open implementation issues, several of which are demonstrated to be genuine compile-time or run-time defects rather than mere stylistic concerns.

Contents

1 Introduction and Motivation **3**

2 Type-Constraint System **3**
- 2.1 `is_bigint`: An Open Extension Point . . . 3
- 2.2 `IsAnyOf` and `AnyStandardString` . . . 4
- 2.3 `MemVarType`: The Master Constraint . . . 4
- 2.4 Stream Insertion Overloads for Non-Narrow Strings . . . 5

3 The `memvar<T>` Class **5**
- 3.1 Internal Representation . . . 5
- 3.2 Construction and the Rule-of-Five Profile . . . 6
- 3.3 Value Access . . . 7
- 3.4 Mutation . . . 7
- 3.5 History Management . . . 8
- 3.6 Printing . . . 9

4 The `memvarTimed<T, Time, Clock>` Class **9**
- 4.1 Additional Storage . . . 9
- 4.2 Construction . . . 9
- 4.3 The Overridden `setValue` and a Subtlety in `clearHistory` . . . 10
- 4.4 Time-Tagged Access and Printing . . . 11

5 Free Functions and Namespace-Scope Operator Overloads **11**
- 5.1 Comparison and Arithmetic Operators . . . 11

5.2 Stream Insertion and Template Partial Ordering 11
5.3 Free `getHistoryValue` / `getHistoryTimedValue` Helpers 12

6 Complete Usage Examples **13**
6.1 Basic Numeric `memvar` 13
6.2 Postfix vs. Prefix Semantics, Strings, and Exceptions 14
6.3 `memvarTimed` and Polymorphic Use 14
6.4 The `Time`/`Clock` Template-Parameter Defect (Confirmed) 15

7 Edge Cases **16**
7.1 The `Time` Template Parameter Is Effectively Non-Functional for Coarser Granularities . 16
7.2 The Free `getHistoryTimedValue` and Timed `operator«` Silently Assume Default `Time`/`Clock` 17
7.3 Backspace-Based Trailing-Separator Erasure Is Stream- and Separator-Length-Dependent 17
7.4 `const`/`volatile`-Qualified `T` Passes the Concept but Breaks the Class Body 17
7.5 `char` (and Other Narrow/Wide Character Types) Are Admitted as "Integral" 18
7.6 The Tuple "Error Flag" Is Vestigial 18
7.7 Copy-Assignment Copies Only the Current Value, Never the History or Capacity 18
7.8 Prefix/Postfix Increment Depend on the Minimum History Capacity Invariant 19
7.9 No Protection Against Division by Zero or Arithmetic Overflow 19
7.10 Thread Safety 19

8 Limitations, Constraints, and Open Issues **19**
8.1 Compile-Time / Template-System Limitations 19
8.2 Runtime Behavior and Correctness 20
8.3 API Design Constraints 20
8.4 Non-Functional Constraints 21

9 Conclusion **21**

1 Introduction and Motivation

`memvar.h` is a self-contained, header-only C++20 library (`#pragma once`, no separate translation unit) that defines a namespace `mv` containing:

- A primary class template `memvar<T>`: a value of type `T` that remembers a bounded number of its previous values in a first-in/first-out history buffer.
- A derived, `final` class template `memvarTimed<T, Time, Clock>`: the same behavior as `memvar<T>`, but with each historical value additionally tagged with the time (relative to an internally recorded epoch) at which it was set.
- A supporting type-constraint system (`is_bigint`, `IsAnyOf`, `AnyStandardString`, `MemVarType`) that restricts which types `T` may legally instantiate these templates.
- A set of free (namespace- and global-scope) operator overloads — comparison, arithmetic, and stream insertion — that allow `memvar` and `memvarTimed` objects to interoperate with their underlying value type `T` and with `std::ostream` largely as if they were plain values of type `T`.
- Four global `operator«` overloads for `std::wstring`, `std::u8string`, `std::u16string`, and `std::u32string`, which exist because the standard library provides no such overloads and the library's own printing routines rely on `operator«` being defined for whatever `T` is chosen.

From the structure of the code, the underlying motivation is to provide a "smart" scalar/string variable — something that behaves, syntactically, almost exactly like a plain `int`, `double`, or `std::string` (via an implicit conversion operator, arithmetic compound-assignment operators, comparison operators, increment/decrement, and stream insertion), while transparently recording a rolling window of its past values. The `memvarTimed` variant extends this to also record *when* each value was assigned, which is useful for lightweight instrumentation, debugging, telemetry sampling, or auditing of variable mutation over time, without requiring the calling code to maintain any history bookkeeping of its own.

The design constrains `T` to a closed (but nominally extensible) set of categories: built-in integral types (excluding `bool`), built-in floating-point types, the five standard string types, and a single external big-integer type (`bip::bigint`), reachable only via a forward declaration and a trait specialization. This suggests the library is intended for numeric/textual telemetry variables rather than as a general-purpose "observable value" container for arbitrary user types.

2 Type-Constraint System

2.1 `is_bigint`: An Open Extension Point

```
template <typename T>
struct is_bigint : std::false_type {};

template <>
struct is_bigint<bip::bigint> : std::true_type {};

template <typename T>
inline constexpr bool is_bigint_v = is_bigint<T>::value;
```

This is the classical C++ trait-specialization pattern: the primary template says "false" for every type, and a single explicit specialization flips it to "true" for `bip::bigint`, a type that is only *forward-declared* (`namespace bip { class bigint; }`) at the top of the file, never defined. Because `is_bigint` is a class template that can be specialized by any translation unit, users of the header could, in principle, add further specializations (e.g., `is_bigint<my::int128>`) to admit additional "big integer"-like types into `memvar`, provided such a specialization is visible before the point of instantiation. However, admission

through this trait says nothing about whether the type actually supports the operations (`operator+`, `operator-`, `operator*`, `operator/`, relational operators, `operator<<`) that `memvar`'s member and free-function operators assume; the trait only gates entry into the `MemVarType` concept, and any missing operation on the concrete type will only surface as a (typically verbose) hard compile error at the point where the missing operator is used, not at the point of gating.

2.2 `IsAnyOf` and `AnyStandardString`

```
template <typename T, typename... Types>
concept IsAnyOf = (std::same_as<T, Types> || ...);

template <typename T>
concept AnyStandardString = IsAnyOf<std::remove_cvref_t<T>,
                                    std::string,
                                    std::wstring,
                                    std::u8string,
                                    std::u16string,
                                    std::u32string>;
```

`IsAnyOf` is a generic, reusable fold-expression concept testing whether `T` is exactly one of a list of candidate types (after the caller applies whatever cv/reference stripping it wants). `AnyStandardString` instantiates it against `std::remove_cvref_t<T>` and the five standard string specializations that ship with the C++ standard library, including the C++20 addition `std::u8string`. This concept is used pervasively as a negative constraint (`requires (!AnyStandardString<T>)`) to disable arithmetic operators that make no sense for strings, such as subtraction, multiplication, division, and increment/decrement.

2.3 `MemVarType`: The Master Constraint

```
template <typename T>
concept MemVarType = (std::is_integral_v<std::remove_cvref_t<T>> &&
                      !std::same_as<std::remove_cv_t<T>, bool>) ||
                      std::is_floating_point_v<std::remove_cvref_t<T>> ||
                      AnyStandardString<T> ||
                      is_bigint_v<std::remove_cvref_t<T>>;
```

`MemVarType<T>` is satisfied when `T`, after removing cv-qualifiers and references, is: an integral type other than `bool`; *or* a floating-point type; *or* one of the standard string types; *or* a type recognized by `is_bigint`. This concept is used as the constraint on the template parameter of `memvar` itself:

```
template <MemVarType T>
class memvar { /* ... */ };
```

Two consequences of this formulation are noteworthy (and are revisited in Section 7):

- `bool` is explicitly excluded, but every *other* built-in integral type is admitted, including `char`, `signed char`, `unsigned char`, `char8_t`, `char16_t`, `char32_t`, and `wchar_t`. A `memvar<char>` is legal and will print via the character overload of `operator<<(std::ostream&, char)`, not as a numeric code point.
- The concept strips cv-qualifiers and references *only for the purpose of the check*. The bare, unstripped `T` is what is actually used everywhere inside the class body (e.g. `std::deque<T> memo_`). A type such as `const int` passes the concept (its stripped form, `int`, is integral and not `bool`) but breaks the class body, because `std::deque` statically asserts that its value type must be non-const, non-volatile.

2.4 Stream Insertion Overloads for Non-Narrow Strings

```
inline std::ostream& operator<<(std::ostream& os, const std::wstring& s) {
  std::wstring_convert<std::codecvt_utf8<wchar_t>> conv;
  return os << conv.to_bytes(s);
}
inline std::ostream& operator<<(std::ostream& os, const std::u8string& s) {
  return os << reinterpret_cast<const char*>(s.c_str());
}
inline std::ostream& operator<<(std::ostream& os, const std::u16string& s) {
  std::wstring_convert<std::codecvt_utf8_utf16<char16_t>, char16_t> conv;
  return os << conv.to_bytes(s);
}
inline std::ostream& operator<<(std::ostream& os, const std::u32string& s) {
  std::wstring_convert<std::codecvt_utf8<char32_t>, char32_t> conv;
  return os << conv.to_bytes(s);
}
```

These four overloads exist at global scope (outside `namespace mv`) because `std::ostream` has no built-in `operator«` for any of `std::wstring`, `std::u8string`, `std::u16string`, or `std::u32string`, yet `memvar`'s printing routines (Section 3.6) call `os « item` for whatever `T` was chosen. Three of the four overloads route through `std::wstring_convert`, a UTF conversion facility that was deprecated in C++17 with no replacement standardized as of C++20/23; the `std::u8string` overload instead performs a `reinterpret_cast` of the `char8_t` buffer to `const char*`, relying on the fact that `char8_t` and `char` share the same size and (UTF-8) representation.

Reliance on Deprecated Library Facilities

Three of the four Unicode stream-insertion overloads depend on `std::wstring_convert` and the associated `std::codecvt_utf8`/`std::codecvt_utf8_utf16` facets, all deprecated since C++17. GCC 13 compiles this code only with deprecation warnings (suppressed by default, visible with `-Wdeprecated-declarations`); a future standard library that removes these facilities entirely will break compilation of `memvar<std::wstring>`, `memvar<std::u16string>`, and `memvar<std::u32string>` (but not `memvar<std::u8string>`, whose overload avoids `wstring_convert`).

3 The `memvar<T>` Class

3.1 Internal Representation

The internal state of a `memvar<T>` object consists of exactly three data members:

```
using capacityType = int64_t;
using memvarHistory = std::deque<T>;

static constexpr capacityType minimumHistoryCapacity_ {2};
static constexpr capacityType historyCapacityDefault_ {10};
const capacityType historyCapacity_ {historyCapacityDefault_};

memvarHistory memo_ {};
```

- `historyCapacity_` is a *const* data member fixed for the lifetime of the object, set either to the compile-time default of 10 or to a value supplied at construction. Because it is `const`, it can never be changed after construction (not even by copy-assignment, which does not touch it), and because `memvar`'s copy constructor is deleted, the only way to obtain a `memvar` with a different capacity is to construct a new object.

- `memo_` is a `std::deque<T>` used as a bounded FIFO: new values are pushed at the front (`emplace_front`) and, once the size exceeds `historyCapacity_`, the oldest value is discarded from the back (`pop_back`). Index `0` is therefore always the *current* (most recently set) value, and increasing indices walk backwards through history, oldest value at index `size()-1`.
- `capacityType` is a signed 64-bit integer, explicitly documented (in code, via its own name and the type chosen) to be signed — this signedness is exploited by `getHistoryValue`'s bounds check (`index >= 0`) and by the reverse-iteration loop in `memvarTimed::printReverseHistoryTimedData`, which counts a loop variable down through zero and would infinite-loop or wrap around if it were unsigned.

A `std::deque` was chosen over, e.g., a `std::vector` or a fixed-size ring buffer almost certainly because it provides amortized O(1) insertion and removal at *both* ends without invalidating references to other elements (aside from the removed one), which is exactly the access pattern required (push front, pop back) by a bounded rolling history.

3.2 Construction and the Rule-of-Five Profile

```
memvar() {
  memo_.emplace_front(T{});
}

explicit memvar(const T& value,
                const capacityType historyCapacity = historyCapacityDefault_)
                   :
historyCapacity_ (historyCapacity) {
  if ( historyCapacity_ < minimumHistoryCapacity_ ) {
    throw std::invalid_argument("ERROR: The history capacity must be "
      + std::to_string(minimumHistoryCapacity_) + " at least, or more");
  }
  memo_.emplace_front(value);
}

virtual ~memvar() = default;

memvar(const memvar& rhs) = delete;
memvar(memvar&& rhs) = delete;
memvar& operator=(memvar&& rhs) = delete;
```

The default constructor leaves `historyCapacity_` at its default (10, which trivially satisfies the minimum-capacity check, so no validation is performed there) and seeds the history with a single default-constructed value of `T` (`0` for arithmetic types, the empty string for standard strings). The single-argument/two-argument constructor is `explicit` (blocking implicit conversions such as passing a bare `T` where a `memvar<T>` is expected) and validates that the requested capacity is at least `minimumHistoryCapacity_` (2), throwing `std::invalid_argument` otherwise. The destructor is `virtual`, enabling safe polymorphic deletion through a `memvar<T>*` or `memvar<T>&` that actually refers to a `memvarTimed<T,...>` object.

The special-member-function profile is deliberately asymmetric:

	Default	Copy	Move
Construction	user-defined	**deleted**	**deleted**
Assignment	(n/a)	user-defined (kept)	**deleted**

This means a `memvar<T>` object can never be copy-constructed or move-constructed/assigned, but *can* be copy-assigned. This asymmetry is examined further in Section 7.

3.3 Value Access

```
operator T() const {            // implicit conversion to T
  return getValue();
}
T operator()() const {          // current value
  return getValue();
}
T operator()(const capacityType index) const {   // historical value by index
  return std::get<T>(getHistoryValue(index));
}
T operator[](const capacityType index) const {   // identical to operator()(
    index)
  return std::get<T>(getHistoryValue(index));
}
```

together with the protected/public accessors:

```
protected:
  const T& getValue() const { return memo_.at(0); }
public:
  historyValue getHistoryValue(const capacityType index) const {
    if ( (index < static_cast<capacityType>(memo_.size())) && (index >= 0) ) {
      return std::make_tuple(memo_.at(static_cast<size_t>(index)), false);
    }
    throw std::out_of_range( /* ... */ );
  }
  auto getHistoryMinMax() const {
    const auto [min, max] = std::ranges::minmax_element(memo_);
    return std::make_tuple(*min, *max);
  }
  const auto& getMemVarHistory() const noexcept { return memo_; }
```

An implicit conversion operator to `T` allows a `memvar<T>` to be used almost anywhere a plain `T` is expected (e.g. `int n = my_memvar;`). `operator()` with no arguments and `operator()(index)`/`operator[](index)` provide, respectively, the current value and an arbitrary historical value by index; the latter two are functionally identical, differing only in call syntax (`m(2)` versus `m[2]`). `getHistoryValue` returns a 2-tuple `(T value, bool errorFlag)`; as shown by the code, the `bool` is *always* constructed as `false` on every successful return, and the only error path (out-of-range index) throws an exception rather than returning a tuple with the flag set to `true` (Section 7 discusses this as a vestigial/dead field). `getHistoryMinMax` returns the minimum and maximum values currently held in the history, computed with `std::ranges::minmax_element`, requiring only that `T` support `operator<`.

3.4 Mutation

```
virtual void setValue(const T& value) {
  memo_.emplace_front(value);
  if ( static_cast<capacityType>(memo_.size()) > historyCapacity_ ) {
    memo_.pop_back();
  }
}
```

`setValue` is the single choke point through which every mutation of a `memvar` passes: it is `virtual` and `protected`, and is overridden by `memvarTimed` (Section 4) to additionally record a time tag. Every public mutator — `operator=(const T&)`, `operator=(const memvar&)`, `operator+=`, `operator-=`, `operator*=`, `operator/=`, and the increment/decrement family — is implemented purely in terms of `getValue()` and `setValue()`:

```
memvar& operator=(const T& rhs) { setValue(rhs); return *this; }
memvar& operator=(const memvar& rhs) { setValue(rhs.getValue()); return *this;
    }
memvar& operator+=(const T& rhs) { setValue(getValue() + rhs); return *this; }
memvar& operator-=(const T& rhs) requires (!AnyStandardString<T>) {
  setValue(getValue() - rhs); return *this;
}
// *=, /= follow the same pattern, both string-excluded
```

`operator+=` is available unconditionally (strings support concatenation via `operator+`); `operator-=`, `operator*=`, and `operator/=` are constrained with `requires (!AnyStandardString<T>)` and are simply absent from overload resolution for any of the five string types. Increment and decrement are implemented via two private helper pairs:

```
T incr1() requires (!AnyStandardString<T>) {
  const T newValue {getValue() + static_cast<T>(1)};
  setValue(newValue);
  return newValue;
}
void voidIncr1() requires (!AnyStandardString<T>) {
  setValue(getValue() + static_cast<T>(1));
}
// decr1 / voidDecr1 mirror incr1 / voidIncr1 with subtraction

T operator++() requires (!AnyStandardString<T>) {              // ++m
  return incr1();
}
T operator++([[maybe_unused]] int dummy) requires (!AnyStandardString<T>) {
    // m++
  voidIncr1();
  return std::get<T>(getHistoryValue(1));
}
```

Prefix `++` pushes the incremented value and returns it directly. Postfix `m++` pushes the incremented value (via `voidIncr1`, which discards the return of `setValue`) and then *re-reads* the value now sitting at history index 1 — i.e. the value that was current *before* the increment, since the new value occupies index 0 after the push. This is correct postfix semantics (return the pre-increment value), but its correctness depends on the history always containing at least two elements immediately after a push, which is guaranteed only because `minimumHistoryCapacity_` is 2 and is enforced by the throwing constructor; Section 7 examines this dependency explicitly.

3.5 History Management

```
capacityType getHistorySize() const noexcept {
  return static_cast<capacityType>(memo_.size());
}
virtual void clearHistory() {
  memo_.clear();
  setValue(T{});
}
auto isHistoryFull() const noexcept {
  return static_cast<capacityType>(memo_.size()) >= historyCapacity_;
}
```

`clearHistory` is `virtual` (overridden by `memvarTimed`) and resets the object to a single default-constructed value, i.e. it does not leave the history empty, it leaves it with exactly one element (the same convention as the constructors). `getHistorySize` and `isHistoryFull` are simple, `noexcept` queries over `memo_.size()` versus `historyCapacity_`.

3.6 Printing

```
void memvarPrinter (const memvarHistory& history,
                    std::ostream& os = std::cout,
                    const bool printReverse = false,
                    const std::string& separator = std::string(" ")) const {
  if ( history.empty() ) { os << "[]\n"; return; }

  auto printItem = [&separator, &os] (const T& item) -> void {
    os << item << separator;
  };

  os << "[ ";
  if ( printReverse ) { std::for_each(history.crbegin(), history.crend(),
      printItem); }
  else                { std::for_each(history.cbegin(),  history.cend(),
      printItem); }
  os << "\b \b ]\n";
}
```

`printHistoryData` and `printReverseHistoryData` both delegate to this single private helper, differing only in iteration direction (forward from newest-to-oldest, or reverse from oldest-to-newest). Every printed item is followed by the `separator` string (a single space by default); after the loop, the sequence `"\b \b ]\n"` is written — a *backspace, space, backspace* sequence intended to erase the trailing separator before printing the closing bracket. This technique, its assumptions, and its failure modes are examined in Section 7.

4 The `memvarTimed<T, Time, Clock>` Class

```
template <MemVarType T,
          typename Time = std::chrono::nanoseconds,
          typename Clock = std::chrono::steady_clock>
class memvarTimed final : public memvar<T> {
  /* ... */
};
```

`memvarTimed` publicly inherits from `memvar<T>` and adds two independently templated parameters, `Time` (the duration type used to tag each historical value; defaults to `std::chrono::nanoseconds`) and `Clock` (the clock used to obtain timestamps; defaults to `std::chrono::steady_clock`). The class is marked `final`, so it cannot itself be further subclassed.

4.1 Additional Storage

```
private:
  using memvarTimeHistory = std::deque<std::chrono::time_point<Clock, Time>>;
  memvarTimeHistory timeMemo_ {};
  std::chrono::time_point<Clock, Time> memvarEpoch_ {};
```

Two members are added: `timeMemo_`, a second deque mirrored in lock-step with the base class's `memo_` (same push-front / pop-back discipline, same capacity), and `memvarEpoch_`, the time point recorded at construction (or at the most recent `clearHistory()`) against which all reported time tags are measured.

4.2 Construction

```
memvarTimed() : memvar<T>() {
  memvarEpoch_ = Clock::now();
```

```
  timeMemo_.emplace_front(memvarEpoch_);
}
explicit memvarTimed(const T& value,
                     const memvar<T>::capacityType historyCapacity = memvar<T
                         >::historyCapacityDefault_) :
memvar<T>(value, historyCapacity) {
  memvarEpoch_ = Clock::now();
  timeMemo_.emplace_front(memvarEpoch_);
}
```

Both constructors first delegate to the corresponding `memvar<T>` base constructor (which populates `memo_` directly, *not* through the virtual `setValue`), and only afterwards initialize `memvarEpoch_` and push the first entry into `timeMemo_`. Because base-class construction happens before the derived object's virtual table is fully "switched over" to `memvarTimed`, and because the base constructors do not call any virtual function, this design correctly sidesteps the classic C++ pitfall of relying on virtual dispatch during construction.

4.3 The Overridden `setValue` and a Subtlety in `clearHistory`

```
private:
  void setTimeTag() { timeMemo_.emplace_front(Clock::now()); }

  void setValue(const T& value) override {
    setTimeTag();
    if ( static_cast<memvar<T>::capacityType>(timeMemo_.size()) > memvar<T>::
        historyCapacity_ ) {
      timeMemo_.pop_back();
    }
    memvar<T>::setValue(value);
  }
```

`memvarTimed` overrides `setValue` *privately*. This is legal C++: access specifiers control name lookup at the call site, not virtual dispatch, so any call to `setValue` on an object whose dynamic type is `memvarTimed` — including calls made from *within* `memvar<T>`'s own member functions — will invoke this override, regardless of the override's access level. The override records a new time tag, trims `timeMemo_` to capacity, and then calls the base class's `setValue` *via the qualified name* `memvar<T>::setValue(value)`, which suppresses virtual dispatch for that one call and thus avoids infinite recursion.

```
void clearHistory() override {
  memvar<T>::clearHistory();
  timeMemo_.clear();
  memvarEpoch_ = Clock::now();
  timeMemo_.emplace_front(memvarEpoch_);
}
```

This override calls `memvar<T>::clearHistory()` with an explicit qualification, which suppresses virtual dispatch for the call to `clearHistory` itself. However, *inside* `memvar<T>::clearHistory()`, the statement `setValue(T{})` is an ordinary (unqualified) call on `*this`; because `setValue` is virtual and the dynamic type of `*this` is `memvarTimed`, this call *does* dispatch virtually to `memvarTimed::setValue`, which pushes a spurious time tag into `timeMemo_` (immediately followed, back in `memvarTimed::clearHistory`, by `timeMemo_.clear()`, which discards it). The net externally observable state is correct — both `memo_` and `timeMemo_` end up with exactly one, mutually consistent entry — but the intermediate work (an extra `Clock::now()` call and two extra deque operations) is unnecessary and stems directly from a base-class member function reaching a derived override through ordinary virtual dispatch, a subtlety that is easy to overlook when reading `memvar<T>::clearHistory` in isolation.

4.4 Time-Tagged Access and Printing

```
Time getTimeTag(const size_t index = 0) const {
  return std::chrono::duration_cast<Time>(timeMemo_.at(index) - memvarEpoch_);
}
historyTimedValue getHistoryTimedValue(const memvar<T>::capacityType index)
    const {
  return std::make_tuple(std::get<0>(memvar<T>::getHistoryValue(index)),
                         getTimeTag(static_cast<size_t>(index)), false);
}
```

`getTimeTag(index)` returns the duration between the epoch and the time point at the given history index, cast into the `Time` duration type. `getHistoryTimedValue` combines a historical value (obtained via the base class's `getHistoryValue`) with its time tag into a 3-tuple `(T, Time, bool)`; as with the base class's tuple, the trailing `bool` is unconditionally `false`.

`printHistoryTimedData` and `printReverseHistoryTimedData` iterate `memvar<T>::memo_` (accessed directly, since `memvarTimed` is a subclass and `memo_` is `protected`) alongside `getTimeTag(i)`, printing bracketed `[time:value]` pairs. Notably, the reverse-printing loop uses a *signed* loop counter explicitly so that the downward-counting `for` loop can test `i >= 0` and terminate correctly at index 0, rather than wrapping around as an unsigned counter would.

5 Free Functions and Namespace-Scope Operator Overloads

5.1 Comparison and Arithmetic Operators

For each of ==, >, <, >=, <=, +, -, *, and /, the header defines three free-function templates: `memvar` vs. `memvar`, `memvar` vs. `T`, and `T` vs. `memvar`, e.g.:

```
template <typename T>
bool operator==(const mv::memvar<T>& mv1, const mv::memvar<T>& mv2) {
  return mv1() == mv2();
}
template <typename T>
bool operator==(const mv::memvar<T>& mv, const T& v) { return mv() == v; }
template <typename T>
bool operator==(const T& v, const mv::memvar<T>& mv) { return v == mv(); }
```

`operator!=` is not defined explicitly; under C++20's rewritten-candidate rules, it is synthesized automatically from `operator==`. `operator-`, `operator*`, and `operator/` are constrained with `requires (!mv::AnyStandardString<T>)`, consistent with the member compound-assignment operators; `operator+` is not string-restricted, since `std::string` supports concatenation. All of these free functions take their operands *by reference* to `memvar<T>` but *return `T` by value*, meaning arithmetic between two `memvars` or between a `memvar` and a `T` produces a plain `T`, not a `memvar<T>`; the history is never touched by these operators (only by `operator+=`, `operator-=`, etc., or explicit assignment).

Because these are templates taking a reference-to-`memvar<T>` (or reference-to-`memvarTimed<T,...>` through the base-class relationship), argument-dependent lookup and base-class template deduction allow the very same free functions to be used with `memvarTimed` operands without any additional overloads: a `memvarTimed<T>` argument binds to a `const mv::memvar<T>&` parameter and `T` is deduced from the derived-to-base relationship.

5.2 Stream Insertion and Template Partial Ordering

```
template <typename T>
std::ostream& operator<<(std::ostream& os, const mv::memvar<T>& mv) {
```

```
  return os << mv();
}
template <typename T>
std::ostream& operator<<(std::ostream& os, const mv::memvarTimed<T>& mvt) {
  return os << "[" << mvt.getTimeTag().count() << ":" << mvt() << "]";
}
```

Two overloads of `operator«` are defined: a generic one for any `memvar<T>` (prints just the current value), and a more specialized one for `memvarTimed<T>` (prints a `[time:value]` pair). Both are viable candidates when streaming a `memvarTimed<T>` object, but C++20's partial-ordering rules for function templates select the more specialized overload (the one taking `memvarTimed<T>` directly, rather than the one taking its base `memvar<T>`) automatically and unambiguously, so `std::cout « my_timed_var;` prints the timed form without any explicit disambiguation on the caller's part.

Limitation of the Timed Stream-Insertion Overload

The `operator«` overload for `memvarTimed<T>` is written as `const mv::memvarTimed<T>&`, which — exactly as discussed in Section 5.3 below for `getHistoryTimedValue` — silently fixes the `Time` and `Clock` parameters to their class-template defaults (`std::chrono::nanoseconds` and `std::chrono::steady_clock`). A `memvarTimed` instantiated with a non-default `Time` or `Clock` does not match this overload at all, and streaming such an object falls back silently to the generic, non-timed `operator«(std::ostream&, const mv::memvar<T>&)`, printing only the bare value with no diagnostic that the "timed" formatting was skipped.

5.3 Free `getHistoryValue` / `getHistoryTimedValue` Helpers

```
template <typename T>
T getHistoryValue(const memvar<T>& mv, const typename memvar<T>::capacityType
    index) {
  return std::get<T>(mv.getHistoryValue(index));
}

template <typename T>
T getHistoryTimedValue(const memvarTimed<T>& mvt, const typename memvar<T>::
    capacityType index) {
  return std::get<T>(mvt.getHistoryTimedValue(index));
}
```

These two free templates duplicate the names of existing public member functions (`memvar::getHistoryValue`, `memvarTimed::getHistoryTimedValue`) at namespace scope, unwrapping the tuple down to the bare `T` value. The first is unproblematic: it is templated only on `T`, and `memvar<T>` itself has only one template parameter, so any `memvar<T>` (or, through derived-to-base deduction, any `memvarTimed<T, ...>` regardless of its `Time`/`Clock` arguments) binds correctly.

The second, `getHistoryTimedValue`, is templated *only* on `T`, yet its parameter type is written as `const memvarTimed<T>&`. Because `memvarTimed` has three template parameters with the second and third defaulted, the unqualified name `memvarTimed<T>` inside this free function is shorthand for the one specific specialization `memvarTimed<T, std::chrono::nanoseconds, std::chrono::steady_clock>`. Since `Time` and `Clock` are not parameters of the free function template, they cannot be deduced, and the parameter type is therefore a fixed, concrete class type. This defect was confirmed by compilation: a call such as `mv::getHistoryTimedValue<int>(mt, 0)` where `mt` has type `mv::memvarTimed<int, std::chrono::milliseconds>` fails to find a matching overload, because `memvarTimed<int, milliseconds, steady_clock>` and `memvarTimed<int, nanoseconds, steady_clock>` are two unrelated, independent class-template specializations — there is no derived-to-base relationship between different specializations of the same class template, only between a specialization and its actual (source-level) base

class. This function is therefore usable *only* for `memvarTimed` objects that keep both `Time` and `Clock` at their defaults.

6 Complete Usage Examples

All examples in this section were compiled with `g++ -std=c++20 -Wall -Wextra` against the unmodified header and executed; their output is reproduced verbatim from the actual run.

6.1 Basic Numeric `memvar`

```cpp
#include "memvar.h"
#include <iostream>

int main() {
  mv::memvar<int> counter;                    // default: value=0, capacity=10
  counter = 5;
  counter += 3;
  ++counter;
  counter++;
  --counter;

  std::cout << "current value: " << counter() << "\n";
  std::cout << "as int: " << static_cast<int>(counter) << "\n";
  counter.printHistoryData();
  counter.printReverseHistoryData();

  std::cout << "history[0]: " << counter[0] << ", history[1]: " << counter[1]
      << "\n";
  std::cout << "size: " << counter.getHistorySize()
            << " capacity: " << counter.getHistoryCapacity()
            << " full: " << counter.isHistoryFull() << "\n";

  auto [mn, mx] = counter.getHistoryMinMax();
  std::cout << "min: " << mn << " max: " << mx << "\n";

  mv::memvar<int> b(100, 3);   // custom capacity of 3
  for (int i = 0; i < 5; ++i) { b += 1; }
  b.printHistoryData();
  std::cout << "b size after 5 pushes with capacity 3: " << b.getHistorySize()
       << "\n";
}
```

Output:

```
current value: 9
as int: 9
[ 9 10 9 8 5 0 ^H ^H ]
[ 0 5 8 9 10 9 ^H ^H ]
history[0]: 9, history[1]: 10
size: 6 capacity: 10 full: 0
min: 0 max: 10
[ 105 104 103 ^H ^H ]
b size after 5 pushes with capacity 3: 3
```

Note the literal `^H` bytes (control character `0x08`, backspace) in the printed history: this is the "erase-the-trailing-separator" trick from Section 3.6 discussed further in Section 7. On a real terminal these bytes visually erase the preceding character; here, captured to a file/pipe, they appear as literal control characters. Also note that `b`, constructed with an explicit capacity of 3, never grows past 3 stored values despite 5 increments.

6.2 Postfix vs. Prefix Semantics, Strings, and Exceptions

```cpp
#include "memvar.h"
#include <iostream>

int main() {
  mv::memvar<int> x(10, 2); // minimum allowed capacity
  int pre  = ++x;    // x=11, returns 11
  int post = x++;    // x=12, returns 11 (the pre-increment value)
  std::cout << "pre=" << pre << " post=" << post << " x=" << x() << "\n";

  mv::memvar<std::string> s(std::string("hello"));
  s += std::string(" world");
  s = s() + std::string("!");
  std::cout << "s=" << s() << "\n";
  s.printHistoryData();

  try {
    mv::memvar<int> bad(1, 1);           // capacity below the minimum of 2
  } catch (const std::invalid_argument& e) {
    std::cout << "caught: " << e.what() << "\n";
  }

  try {
    mv::memvar<int> y(1);
    y(50);                               // index out of range
  } catch (const std::out_of_range& e) {
    std::cout << "caught: " << e.what() << "\n";
  }

  mv::memvar<int> z(7);
  z += 1; z += 1;
  z.clearHistory();
  std::cout << "after clear: value=" << z() << " size=" << z.getHistorySize()
      << "\n";
}
```

Output:

```
pre=11 post=11 x=12
s=hello world!
[ hello world! hello world hello ^H ^H ]
caught: ERROR: The history capacity must be 2 at least, or more
caught: Index 50 is out of bounds (size is 1).
after clear: value=0 size=1
```

This confirms: (i) postfix `x++` returns the pre-increment value (11) even though `x` itself is now 12; (ii) `memvar<std::string>` supports `operator+=` and plain assignment built from `operator+` on the underlying `std::string`, with each assignment recorded as a new, independent history entry (i.e. no in-place mutation sharing); (iii) both documented exception paths (`std::invalid_argument` for an under-sized capacity, `std::out_of_range` for an invalid index) fire exactly as implemented; and (iv) `clearHistory()` leaves the object with a single, default-valued (0) entry rather than an empty history.

6.3 `memvarTimed` and Polymorphic Use

```cpp
#include "memvar.h"
#include <iostream>
#include <thread>
#include <chrono>

```

```cpp
int main() {
  mv::memvarTimed<double> t(1.5);
  std::this_thread::sleep_for(std::chrono::milliseconds(5));
  t += 2.5;
  std::this_thread::sleep_for(std::chrono::milliseconds(5));
  t = 100.0;

  std::cout << "value: " << t() << "\n";
  std::cout << "timetag(0): " << t.getTimeTag(0).count() << "ns\n";
  t.printHistoryTimedData();

  mv::memvar<double>& base = t;              // upcast to the base class
  std::cout << "via base: " << base() << "\n";
  base.clearHistory();                       // dispatches virtually to
      memvarTimed::clearHistory
  std::cout << "after base.clearHistory(): value=" << base()
            << " size=" << base.getHistorySize() << "\n";
  t.printHistoryTimedData();
}
```

Output:

```
value: 100
timetag(0): 10165757ns
{ --- begin ---
[TimeTag:Value]
[10165757:100]
[5084437:4]
[0:1.5]
  --- end --- }

via base: 100
after base.clearHistory(): value=0 size=1
{ --- begin ---
[TimeTag:Value]
[0:0]
  --- end --- }
```

This confirms that calling `clearHistory()` through a *base-class reference* (`mv::memvar<double>&`) correctly dispatches, via the virtual function mechanism, to `memvarTimed::clearHistory()`: the timed history is properly reset (`timeMemo_` re-seeded with a fresh epoch) even though the call site has no compile-time knowledge that the object is actually a `memvarTimed`.

6.4 The `Time`/`Clock` Template-Parameter Defect (Confirmed)

The following program compiles and runs successfully, demonstrating that `memvarTimed` *can* be instantiated with a non-default `Clock` (as long as `Time` is left at its default) or with a `Time` type that is at least as fine-grained as the clock's native duration:

```cpp
#include "memvar.h"
#include <iostream>
#include <chrono>

int main() {
  // OK: custom Clock, default Time
  mv::memvarTimed<int, std::chrono::nanoseconds, std::chrono::system_clock> a
      (1);
  a += 1;
  std::cout << "custom Clock, default Time OK: " << a.getTimeTag(0).count() <<
       "\n";
```

```

  // OK: a Time finer-grained than steady_clock's native nanoseconds
  mv::memvarTimed<int, std::chrono::duration<long long, std::pico>> b(1);
  b += 1;
  std::cout << "finer-grained Time OK: " << b.getTimeTag(0).count() << "\n";
}
```

Output:

```
custom Clock, default Time OK: 809
finer-grained Time OK: 590000
```

However, instantiating `memvarTimed` with a *coarser* `Time` type — the seemingly natural use case for the template parameter, e.g. tagging values in whole milliseconds instead of nanoseconds — **fails to compile**:

```
mv::memvarTimed<int, std::chrono::milliseconds> mt(42);   // does not compile
mt += 1;
```

Compiling this produces (excerpted; full output spans dozens of lines through `std::deque`'s internals):

```
memvar.h:336:18: error: no match for 'operator=' (operand types are
  'std::chrono::time_point<steady_clock, duration<long,ratio<1,1000>>>'
  and 'steady_clock::time_point' {aka
  'time_point<steady_clock, duration<long,ratio<1,1000000000>>>'})
    memvarEpoch_ = Clock::now();
    ~~~~~~~~~~~~~^~~~~~~~~~~~~
```

This is examined in full in Section 7.

7 Edge Cases

Each item below was verified by compiling and/or executing a minimal program against the unmodified header.

7.1 The `Time` Template Parameter Is Effectively Non-Functional for Coarser Granularities

As demonstrated in Section 6.4, the assignment `memvarEpoch_ = Clock::now();` (present both in the constructors and in `clearHistory`) requires an *implicit* conversion from `Clock::duration` (the value category returned by `Clock::now()`) to the object's own `time_point<Clock, Time>`. The C++20 standard's `std::chrono::time_point` converting constructor is implicit only when the source duration converts to the target duration *without truncation* (i.e. no information loss can occur purely from the ratio of the two periods). For the default `Clock` (`std::chrono::steady_clock`, whose native duration is nanoseconds on the tested platform), this means `Time` must be nanoseconds or finer (e.g. picoseconds); any coarser `Time` — microseconds, milliseconds, seconds, minutes, or any user duration with a coarser period — fails to compile, with the error surfacing deep inside `memvarTimed`'s constructor and `clearHistory`, not at the template-parameter declaration. In effect, the second template parameter of `memvarTimed`, despite being presented as freely configurable, is usable only in the narrow range *"as fine as, or finer than, the chosen clock's native tick"*, which for the default clock excludes essentially every human-meaningful duration coarser than a nanosecond.

7.2 The Free `getHistoryTimedValue` and Timed `operator«` Silently Assume Default `Time`/`Clock`

Independently of the compile-time defect above, Section 5.3 shows that even the (narrow) set of `memvarTimed` instantiations that *do* compile with a non-default `Time` or `Clock` cannot be used with the free function `mv::getHistoryTimedValue<T>(...)`, because that function's parameter type silently pins `Time` and `Clock` to their class-template defaults. The same applies to the dedicated `operator«` overload for `memvarTimed<T>`: such objects silently fall back to the un-timed, base-class `operator«`, with no compiler warning that the more specific, timed overload was never a candidate.

7.3 Backspace-Based Trailing-Separator Erasure Is Stream- and Separator-Length-Dependent

`memvarPrinter` appends `separator` after every printed element, then emits `"\b \b ]"` once, at the end, to erase the final separator before the closing bracket. This assumes the separator is exactly one character wide and that the destination is a device (typically a terminal) that interprets `\b` as a cursor-back operation rather than storing it as data. Both assumptions were falsified experimentally:

```
mv::memvar<int> v(1);
v += 2; v += 3;
v.printHistoryData(std::cout, ", ");    // two-character separator
```

Piped to a file (as any non-terminal destination, e.g. `std::ofstream`, `std::ostringstream`, or a redirected/piped `std::cout`, would be), the raw bytes produced are:

```
[ 6, 3, 1, ^H ^H ]
```

Two separate defects are visible here. First, because the destination is not an interactive terminal, the two `\b` control bytes are written literally into the output rather than erasing anything, so any consumer of this output as text (a log file, a file comparison, a test assertion) sees stray backspace characters. Second, even on a real terminal where `\b` does move the cursor back, the code only ever emits *one* backspace before the corrective space and closing bracket, which erases only the *last character* of the separator; for the two-character separator `", "` used above, this erases only the trailing space, leaving a stray comma immediately before the closing bracket (visually: `[ 6, 3, 1, ]`, not the intended `[ 6, 3, 1 ]`). The erasure logic is correct only for the specific case of a one-character separator printed to an interactive terminal.

7.4 `const`/`volatile`-Qualified `T` Passes the Concept but Breaks the Class Body

`MemVarType<T>` evaluates its integral/floating-point/string/bigint checks against `std::remove_cvref_t<T>`, but the class template itself uses the raw, unstripped `T` throughout its body (`std::deque<T> memo_`, parameters typed `const T&`, etc.). Consequently `mv::memvar<const int>` satisfies `MemVarType` (since `remove_cvref_t<const int>` is `int`) yet fails to compile, because `std::deque` requires a non-`const`, non-`volatile` value type. Confirmed compiler output:

```
.../bits/stl_deque.h:800: error: static assertion failed:
  std::deque must have a non-const, non-volatile value_type
```

The concept therefore does not fully characterize which instantiations of `memvar` are actually usable; it only pre-filters by *category* of type, not by qualification, so a cv-qualified argument is rejected several template layers deeper than the concept check, with a diagnostic that gives no indication that the root cause is a cv-qualifier the user supplied at the `memvar<...>` call site.

7.5 `char` (and Other Narrow/Wide Character Types) Are Admitted as "Integral"

`MemVarType` excludes only `bool` among integral types; `char`, `signed char`, `unsigned char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t` are all integral types in C++20 and are therefore all accepted. A `memvar<char>` compiles and behaves as demonstrated:

```
mv::memvar<char> c('A');
c = 'B';
std::cout << c();    // prints: B
```

This prints the character glyph, not its numeric code point, because `operator«(std::ostream&, char)` is what ultimately gets called. This is standard C++20 behavior rather than a bug in the library, but it is a natural point of surprise for a caller who thinks of `memvar` strictly as "for numbers and strings": `char` silently falls into the "integral" bucket and participates in `operator++`, `operator-`, `operator+=`, etc. with arithmetic (not lexical) semantics, while printing as a character.

7.6 The Tuple "Error Flag" Is Vestigial

Both `historyValue` (`std::tuple<T, bool>`) and `historyTimedValue` (`std::tuple<T, Time, bool>`) reserve a trailing `bool`. Examination of every code path that constructs one of these tuples shows the `bool` is unconditionally `false`:

```
return std::make_tuple(memo_.at(static_cast<size_t>(index)), false);    //
    memvar::getHistoryValue
return std::make_tuple(std::get<0>(memvar<T>::getHistoryValue(index)),
                       getTimeTag(static_cast<size_t>(index)), false); //
                           memvarTimed::getHistoryTimedValue
```

The only failure path for an invalid index throws `std::out_of_range` rather than returning a tuple with the flag set to `true`. From the code alone, there is no route by which a caller ever observes `true` in that position; the field is always present in the type but never carries information, so any code written to branch on it is dead code, and any storage or comparison overhead for the field is pure overhead.

7.7 Copy-Assignment Copies Only the Current Value, Never the History or Capacity

```
memvar& operator=(const memvar& rhs) {
  setValue(rhs.getValue());
  return *this;
}
```

Because `memvar`'s copy constructor is deleted but its copy-assignment operator is not, the only way to obtain a "copy" of a `memvar` is to default-construct one and then assign into it. However, this assignment operator only reads `rhs.getValue()` — the single current value — and pushes it through the assignee's own `setValue`. It does not copy `rhs`'s history, and it cannot copy `rhs`'s `historyCapacity_` (which is `const` on the assignee and was already fixed at the assignee's own construction time). The assignee's pre-existing history (from its own prior lifetime) also remains intact underneath the newly pushed value, rather than being replaced:

```
mv::memvar<int> a(1); a += 2; a += 3;       // a's history: [3,2,1]
mv::memvar<int> b(100);                     // b's history: [100]
b = a;                                      // only copies a's *current* value
    (3)
// b's history is now [3,100], NOT a copy of [3,2,1]
```

A reader accustomed to conventional copy-assignment semantics ("make the left-hand side an indepen-

dent, equal copy of the right-hand side") should not assume that `b` and `a` share any history equivalence after this assignment; only their *current* values compare equal via `operator==`, which itself only compares current values.

7.8 Prefix/Postfix Increment Depend on the Minimum History Capacity Invariant

`operator++(int)` unconditionally reads history index 1 after pushing the incremented value:

```
T operator++([[maybe_unused]] int dummy) requires (!AnyStandardString<T>) {
  voidIncr1();
  return std::get<T>(getHistoryValue(1));
}
```

This is only safe because `minimumHistoryCapacity_` is 2 and is enforced by the explicit constructor's runtime check, guaranteeing the history never shrinks to fewer than 2 elements once it has reached 2. Every constructor path (default, or explicit with a validated capacity ≥ 2) satisfies this invariant, so under the interface *as exposed*, postfix increment/decrement cannot actually observe an out-of-range access. This is correct as implemented, but it is a non-local invariant: the correctness of `operator++(int)` in one part of the file depends on a runtime check performed in the constructor, with no compile-time (e.g. `static_assert`) linkage between the two, so a future edit to either the minimum-capacity constant or the constructor's validation logic could silently reintroduce an out-of-bounds access.

7.9 No Protection Against Division by Zero or Arithmetic Overflow

`operator/=`, the free `operator/`, `incr1/decr1`, and every other arithmetic path perform the underlying `T` operation directly (`getValue() / rhs`, `getValue() + static_cast<T>(1)`, etc.) with no range or zero-divisor checking. For built-in integral `T`, dividing by zero is undefined behavior; for floating-point `T` it produces `inf/nan` per IEEE 754 rather than throwing; incrementing a signed integral type at its maximum representable value is undefined behavior (signed overflow), while incrementing an unsigned type wraps silently. None of these conditions are detected, reported, or guarded against anywhere in the header.

7.10 Thread Safety

No synchronization primitive (mutex, atomic, memory fence) appears anywhere in the header. `memo_`, `timeMemo_`, and `memvarEpoch_` are ordinary, unsynchronized data members backed by `std::deque`, which is not safe for concurrent modification, nor for concurrent read/write, from multiple threads without external locking. Any use of a shared `memvar/memvarTimed` instance across threads requires the caller to supply its own synchronization.

8 Limitations, Constraints, and Open Issues

The following is a consolidated, non-redundant list of every limitation identified in this report, grouped by category.

8.1 Compile-Time / Template-System Limitations

1. `Time` **template parameter of** `memvarTimed` **is unusable for coarser-than-native-clock-tick durations** (Section 7): confirmed to fail to compile for the common case of, e.g., millisecond-granularity time tags with the default clock.
2. **Free** `getHistoryTimedValue` **and the timed** `operator«` **silently assume default** `Time/Clock`, and simply do not match (rather than erroring loudly, in the `operator«` case) for other instantiations.

3. `MemVarType` **concept does not reject cv-qualified** `T` even though the class body cannot actually support it; failures surface deep inside `std::deque`'s internals rather than at the concept boundary.
4. `is_bigint`**-based extensibility gates only the type *category*, not the required operation set**: adding a new "bigint-like" type via specialization does not guarantee that type actually supports every operator `memvar` exposes (`+`, `-`, `*`, `/`, comparisons, `operator«`); missing operations fail only when actually invoked, with instantiation-chain error messages.
5. **Duplicate member/free-function names** (`getHistoryValue`, `getHistoryTimedValue` exist both as member functions and as free functions with different signatures and different return types), which is legal but a potential source of confusion during code review or IDE-assisted navigation.

8.2 Runtime Behavior and Correctness

1. **Backspace-based separator erasure in** `memvarPrinter` is correct only for a one-character separator printed to an interactive terminal; it corrupts output (leaves literal control bytes, and/or a stray trailing separator character) for any non-terminal destination or any separator longer than one character.
2. **Copy-assignment (**`operator=(const memvar&)`**)** copies only the current value, never the source's history or capacity, and layers that single value on top of the destination's own pre-existing history rather than replacing it — behavior that can easily be mistaken for a full, independent copy.
3. **No overflow or division-by-zero protection** anywhere in the arithmetic paths (`operator+=`, `-=`, `*=`, `/=`, `++`, `-`, and the corresponding free operators); behavior for these conditions is whatever the underlying built-in or user-supplied `T` does (UB for signed overflow/integer division by zero, `inf`/`nan` for floating point).
4. **The tuple "error flag" in** `historyValue`**/**`historyTimedValue` **is vestigial**: every construction path sets it to `false`; the actual error-reporting mechanism used throughout the header is exceptions, making the boolean field dead weight in both types.
5. **Redundant work in** `memvarTimed::clearHistory` due to virtual dispatch reaching the derived `setValue` override from within the inherited `clearHistory` body, producing (and immediately discarding) an extra time-tag entry on every call; functionally harmless but wasteful and non-obvious from reading either function in isolation.

8.3 API Design Constraints

1. **History capacity is fixed for the lifetime of an object** (`const` data member) and cannot be changed after construction; changing it requires constructing an entirely new object.
2. **No copy construction or move semantics of any kind**: `memvar` and `memvarTimed` are neither copyable-by-construction nor movable at all (only copy-*assignable*, with the caveats above), which prevents storing these types directly in most standard containers that require at least move construction (e.g. `std::vector<memvar<int»` would need in-place construction via `emplace`-style APIs or indirection through a pointer/`unique_ptr`, since even reallocation-driven moves inside `std::vector` are unavailable).
3. `operator()(index)` **and** `operator[](index)` **are fully redundant**, providing two identical spellings for the same historical-index access with no semantic differentiation.
4. `char` **and other narrow/wide character integral types are silently admitted** by `MemVarType` (only `bool` is excluded), which may not match a caller's intuitive scope of "numeric or string types."
5. **Closed, single-header extensibility for the bigint category**: adding a new admissible "big number"-like type requires a template specialization of `mv::is_bigint` in the same namespace, visible before any instantiation that needs it — there is no runtime or type-erased extension mechanism.

8.4 Non-Functional Constraints

1. **Not thread-safe**: no internal synchronization; concurrent use requires external locking.
2. **Reliance on deprecated standard library facilities** (`std::wstring_convert`, `std::codecvt_utf8`, `std::codecvt_utf8_utf16`) for three of the four non-narrow string `operator«` overloads, with no standardized replacement as of C++20/23; a future standard library that removes these facilities will break compilation for `memvar<std::wstring>`, `memvar<std::u16string>`, and `memvar<std::u32string>`.
3. `operator()` **(value access) always returns** `T` **by value**, incurring a copy on every access; for large `T` (long strings, a hypothetical bigint with many digits) this is a per-call cost with no reference-returning alternative exposed publicly.
4. `std::deque`**'s memory is not guaranteed to shrink** after repeated `pop_back` calls (implementation-defined retention of allocated internal blocks), so the memory footprint of a long-lived, frequently-mutated `memvar` may plateau at its high-water mark rather than tracking its (bounded) logical size precisely.

9 Conclusion

`memvar.h` implements a compact, template- and concept-based value-with-history wrapper for a deliberately restricted set of scalar and string types, with a time-tagged extension built on standard `std::chrono` facilities. Its core mechanics — a `std::deque`-backed bounded FIFO history, a single virtual `setValue` choke-point that both classes' entire mutating interface routes through, and a family of concept-constrained operator overloads that make instances interoperate almost transparently with plain values of type `T` — are internally consistent and, for the primary, default-configured use case (numeric or string variable, default `Time`/`Clock`), behave as the interface suggests, as confirmed by direct compilation and execution. At the same time, the analysis surfaced several concrete, compiler-verified defects rather than purely stylistic concerns: the `Time` template parameter of `memvarTimed` does not work for the great majority of durations a caller would plausibly choose; two free helper functions and the timed stream-insertion operator silently narrow their applicability to the default `Time`/`Clock` pairing without any diagnostic; the type-constraint concept does not fully characterize which instantiations actually compile; and the built-in history-printing routines produce corrupted output outside their one supported scenario (single-character separator, interactive terminal). None of these issues affects the library's most conservative use pattern — a default-configured `memvar<T>` or default-`Time`/`Clock` `memvarTimed<T>` over an arithmetic or string type, value-accessed and printed through the public API's non-parameterized entry points — but they represent meaningful constraints for any usage that departs from that narrow path.